Web-Based Visualization · Part 2

DOM, JavaScript, and D3

Information Visualization · 2026

Today’s agenda

01	**DOM**	Find and change the objects behind a page.
02	**JavaScript**	Read familiar programming ideas in a new syntax.
03	**D3**	Turn three days of Citi Bike data into an SVG chart.

HTML is source text; the DOM is a live object tree

HTML source code is text that describes a page's initial structure and content using tags and attributes.

The **Document Object Model (DOM)** is a programming interface that represents the document as a tree of objects. The browser builds this tree by parsing the HTML.

- **Why use the DOM?** JavaScript needs objects it can find and change: an element's text, attributes, or children.
- Through the DOM, JavaScript changes the current document; the browser updates the displayed page.
- These changes do **not** rewrite the original HTML file.

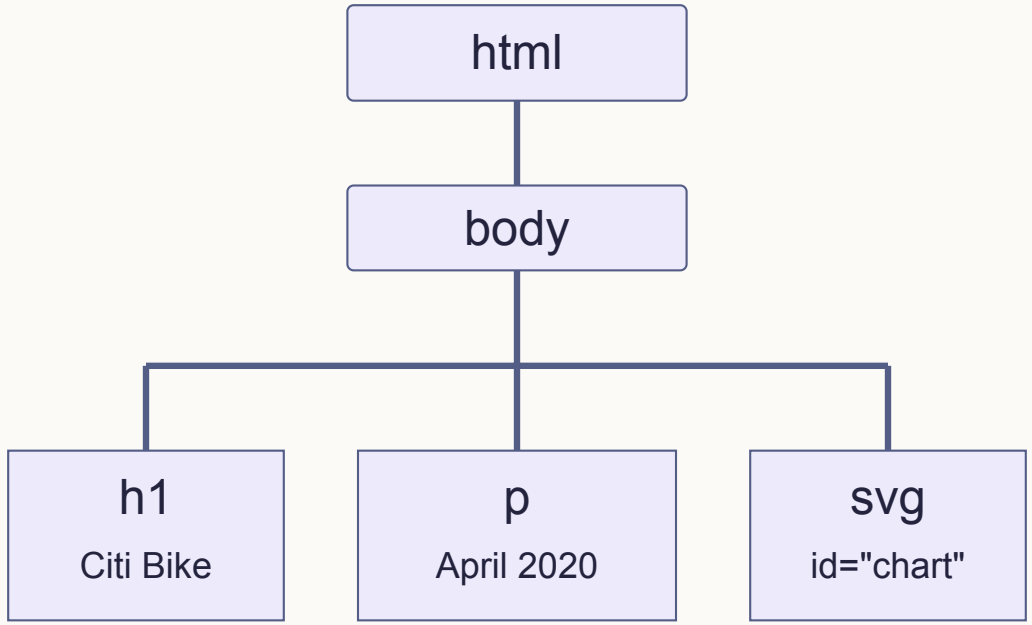
MDN: Document Object Model

The browser turns HTML elements into DOM objects

HTML source text

```
<body>
  <h1>Citi Bike</h1>
  <p>April 2020</p>
  <svg id="chart"></svg>
</body>
```

DOM built by the browser



Each HTML element becomes an object in the tree. JavaScript can find the `h1` object and change its text from “Citi Bike” to “Citi Bike in April”.

querySelector finds a matching element in the DOM

`document.querySelector()` returns the **first element** that matches a CSS selector.

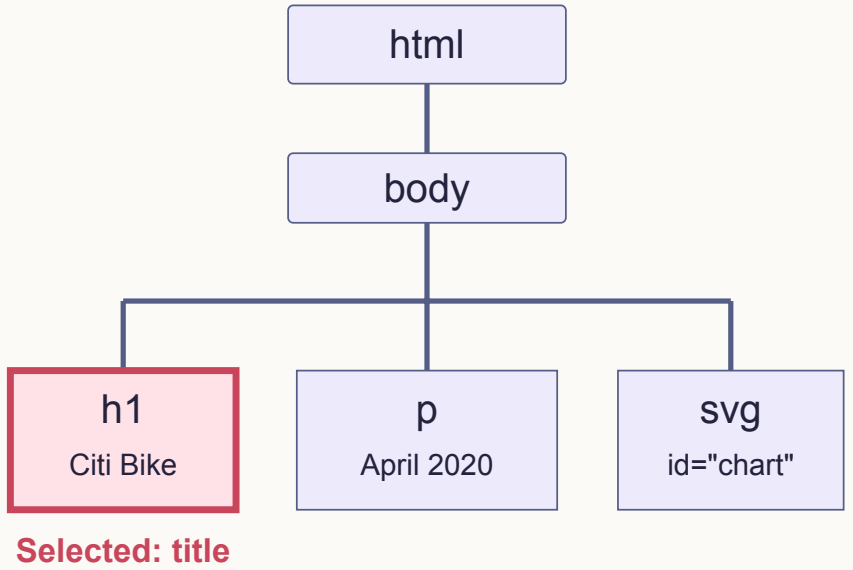
Find the heading

```
const title =
  document.querySelector("h1");
console.log(title.textContent);
```

Console: Citi Bike

`"h1"` matches the heading element from the previous page.

The selected DOM node

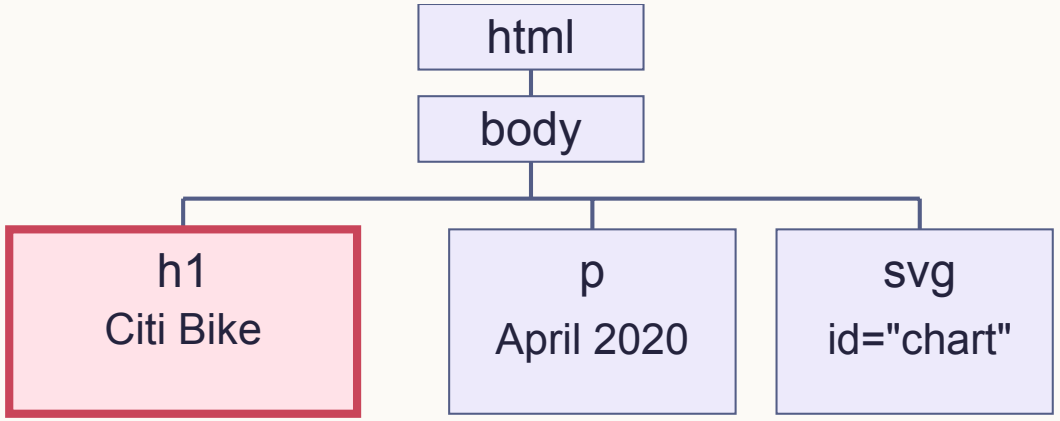


`title` refers to this existing DOM element. JavaScript can now read or change its properties.

Changing the DOM changes the displayed page

```
title.textContent = "Citi Bike in April";
```

Before

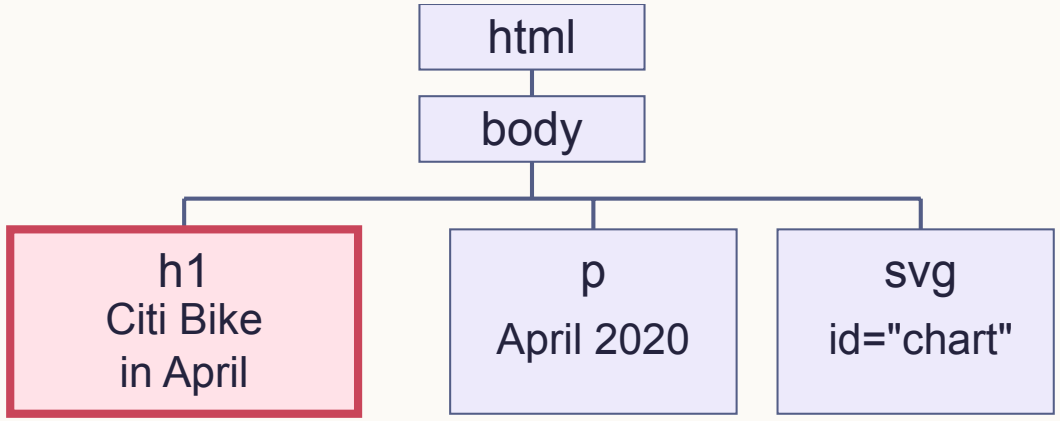


Page display

Citi Bike

April 2020

After



Page display

Citi Bike in April

April 2020

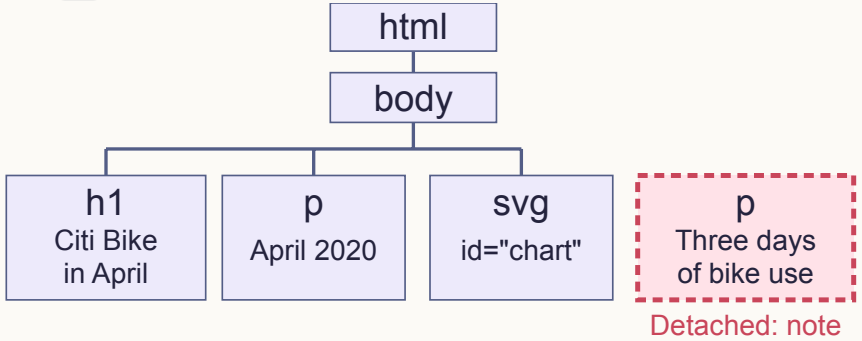
JavaScript changes the `h1` element's text in the DOM; the browser displays the updated heading.

Create the element, then connect it to the DOM

```
const note = document.createElement("p");
note.textContent = "Three days of bike use";
document.body.append(note);
```

1. Create and set the text

The new `p` exists, but is not in the page yet.



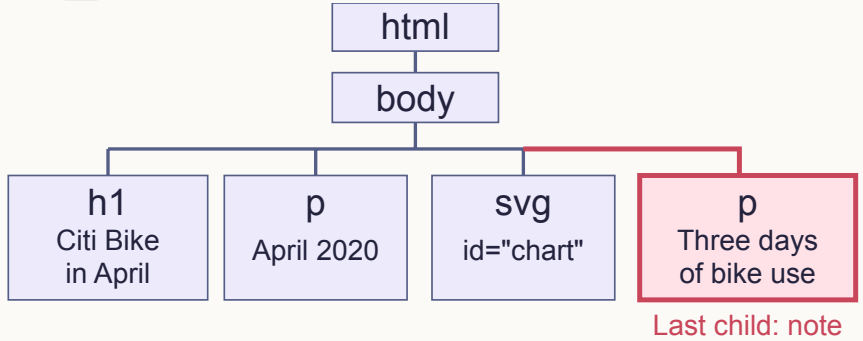
Page display: unchanged

Citi Bike in April

April 2020

2. Append to body

The new `p` becomes body's last child.



Page display: new paragraph appears

Citi Bike in April

April 2020

Three days of bike use

HTML loads JavaScript from a separate file

index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Citi Bike</title>
  <script src="app.js"
          defer></script>
</head>
<body>
  <h1>Citi Bike</h1>
  <p>April 2020</p>
  <svg id="chart"></svg>
</body>
</html>
```

app.js

```
const title =
  document.querySelector("h1");
title.textContent =
  "Citi Bike in April";
const note =
  document.createElement("p");
note.textContent =
  "Three days of bike use";
document.body.append(note);
```

JavaScript is a programming language. Here it changes the DOM to update the page.

Keep both files in the **same folder**. `src="app.js"` loads the code; `defer` runs it after the HTML has been parsed. Open the example ↗

Inspect the updated heading in the console

The browser's **Console** shows messages from JavaScript. Use `console.log()` to inspect a value while your program runs.

Add to the end of app.js

```
console.log(title.textContent);
```

`title` still refers to the same `h1` element. Its text has already changed.

Browser Console output

Console

```
Citi Bike in April
```

>

This message appears in the Console, not in the webpage.

Open the Console: right-click the page → **Inspect** → select the **Console** tab. Reload the page to run `app.js` again and see the message.

Use const by default; let for reassignment

Python

```
rides = 299
rides = rides + 1
```

JavaScript

```
let rides = 299;
rides = rides + 1;
```

```
const city = "Jersey City";
```

`let` permits reassignment. `const` prevents reassignment of that variable; it does not freeze an array or object.

Recognize the values you already know

Meaning	Python	JavaScript
Number	299 / 2.5	299 / 2.5
String	"April"	"April"
Boolean	True / False	true / false
Intentional empty value	None	null
No assigned value	Not a direct equivalent	undefined

JavaScript’s usual numeric type is `number`, for both integers and decimals.

An array stores an ordered sequence

Python

```python
rides = [299, 219, 149]
print(rides[0])
print(len(rides))
```

JavaScript

```javascript
const rides = [299, 219, 149];
console.log(rides[0]);
console.log(rides.length);
```

Both outputs: 299, then 3. Indexing starts at zero.

An object groups named properties

Python

```python
day = {
  "date": "Apr 1",
  "rides": 299
}
print(day["rides"])
```

JavaScript

```javascript
const day = {
  date: "Apr 1",
  rides: 299
};
console.log(day.rides);
```

An object is a collection of named properties. Here `day.rides` and `day["rides"]` both give 299.

A dataset can be an array of objects

```
const daily = [
  {date: "Apr 1", rides: 299},
  {date: "Apr 2", rides: 219},
  {date: "Apr 3", rides: 149}
];
console.log(daily[1].rides); // 219
```

Each object is one daily record; the array preserves their order.

Teaching subset: Jersey City, April 1–3, 2020 · Citi Bike data

Expressions compute and compare values

```
299 + 219        // 518
299 > 250        // true
299 === 299      // true
"299" === 299    // false
"299" + 1        // "2991"
Number("299") + 1 // 300
```

Use `===` for strict equality. `=` assigns a value. Convert numeric text before doing arithmetic.

Conditions choose which code runs

Python

```python
if rides > 250:
    color = "coral"
else:
    color = "steelblue"
```

JavaScript

```javascript
let color;
if (rides > 250) {
  color = "coral";
} else {
  color = "steelblue";
}
```

JavaScript uses parentheses around the condition and braces around each block.

Loop over values with for...of

Python

```python
total = 0
for day in daily:
    total += day["rides"]
print(total)
```

JavaScript

```javascript
let total = 0;
for (const day of daily) {
  total += day.rides;
}
console.log(total);
```

Result: 667 trips across the three dates.

Use an index when position matters

Python

```python
for i in range(3):
    print(i * 60)
```

JavaScript

```javascript
for (let i = 0; i < 3; i++) {
  console.log(i * 60);
}
```

Output: 0, 60, 120 — positions for three rows.

Start at 0 → check `i < 3` → run the block → increment `i`.

A function packages a reusable calculation

Python

```python
def rowY(i):
    return 40 + i * 60
print(rowY(2))
```

JavaScript

```javascript
function rowY(i) {
return 40 + i * 60;
}
console.log(rowY(2));
```

A function accepts inputs (parameters) and can return a result. Here `rowY(2)` returns **160**.

Arrow syntax gives a shorter function

```
const rowY = (i) => {
  return 40 + i * 60;
};
```

For a single returned expression, the braces and `return` can be omitted:

```
const rowY = (i) => 40 + i * 60;
```

Both versions compute the same row position.

A callback is a function passed to another function

```
daily.forEach((day, i) => {
  console.log(day.date, i);
});
```

`forEach()` calls the function once for each array element, in order.

- `day`: the current record.
- `i`: its index — 0, 1, then 2.
- The method supplies these arguments; you choose their names.

map transforms; filter selects

Python

```python
counts = [
  d["rides"] for d in daily
]
busy = [d for d in daily
        if d["rides"] > 250]
```

JavaScript

```javascript
const counts = daily.map(
  d => d.rides
);
const busy = daily.filter(
  d => d.rides > 250
);
```

`counts` is [299, 219, 149]; `busy` contains only the Apr 1 record.

An event runs a callback when something happens

```
<button id="show">Show total</button>
<p id="result">Click the button.</p>
```

```
const button = document.querySelector("#show");
button.addEventListener("click", () => {
document.querySelector("#result").textContent = 667;
});
```

Try the button ↗

Predict the output before running the code

```
const counts = [299, 219, 149];
const busy = counts.filter(n => n > 200);
const labels = busy.map(n => n + " trips");
console.log(labels);
```

- Which values survive the filter?
- What type of value does map return for each item?
- Does counts change?

D3 is a JavaScript library for visualization

D3 is a JavaScript library that provides functions for working with data and creating web visualizations.

- Use ordinary JavaScript to prepare data and repeat operations.
- Use D3 functions to select elements, create SVG shapes, and map values to visual properties.
- The result is ordinary SVG in the DOM, which the browser displays.

D3 documentation

Load the library before your chart program

```
<head>
  <script src="../d3.v7.9.0.min.js" defer></script>
  <script src="chart.js" defer></script>
</head>
<body>
  <svg id="chart" width="520" height="240"></svg>
</body>
```

The first file provides `d3`. The second uses it. Both deferred classic scripts run in document order. SVG canvas: **width 520 × height 240 pixels**.

Select the SVG, then append a rectangle

```
const svg = d3.select("#chart");
const bar = svg.append("rect");
```

- `select()` returns a D3 selection wrapping the matching element.
- `append("rect")` creates a rectangle inside the selected SVG and returns a selection for it.
- The rectangle needs width and height before it becomes visible.

Set SVG attributes with chained calls

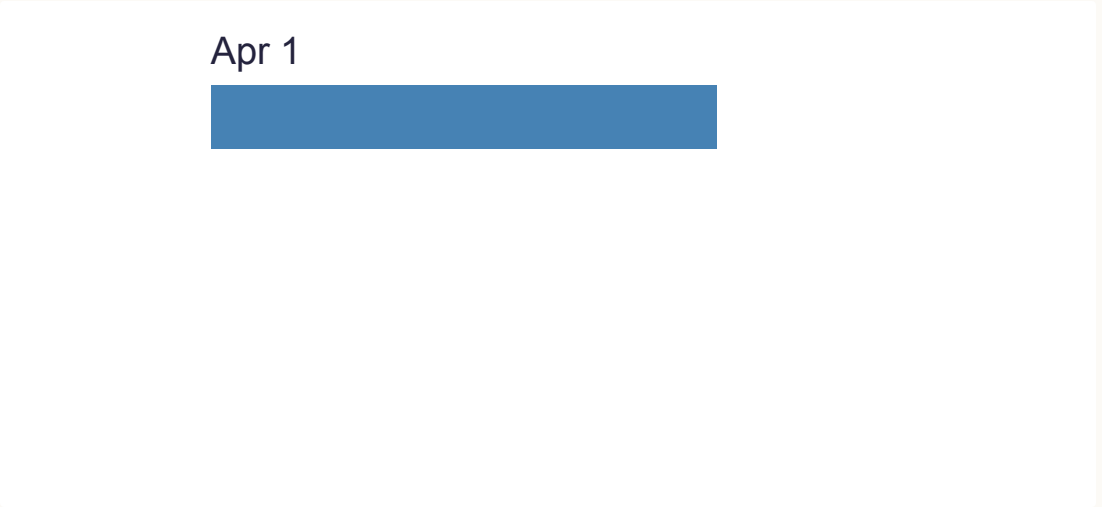
```
bar
  .attr("x", 100)
  .attr("y", 40)
  .attr("width", 240)
  .attr("height", 30)
  .attr("fill", "steelblue");
```



Each `attr()` call returns the selection, so another call can follow.

Add text as another SVG element

```
svg.append("text")
  .attr("x", 100)
  .attr("y", 30)
  .text("Apr 1");
```



`text()` sets the element's text content. `y` locates the text baseline.

A scale maps data values to drawing values

```
const x = d3.scaleLinear()
  .domain([0, 400])
  .range([0, 320]);

console.log(x(200)); // 160
```

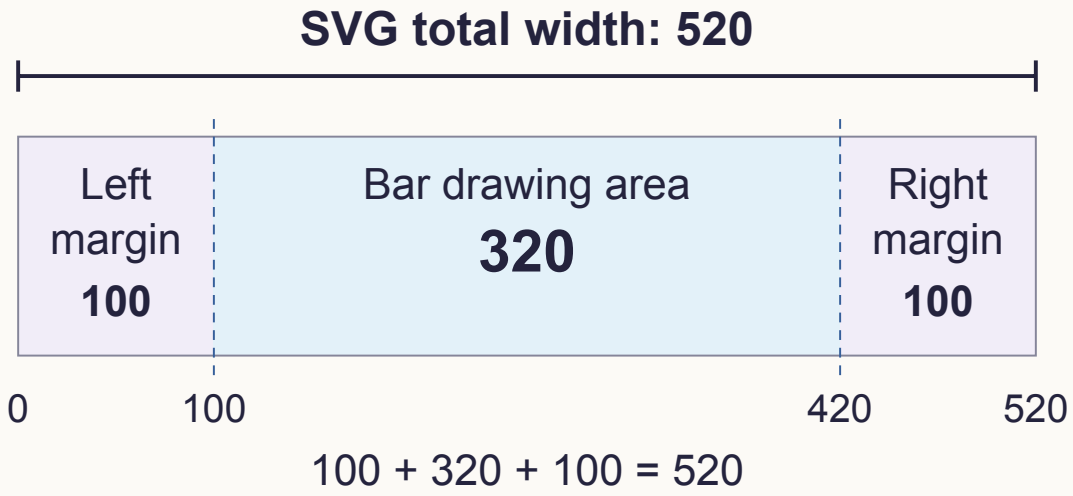
Domain: input values in trips. **Range:** output lengths in pixels.

Why 320? SVG width 520 – left margin 100 – right margin 100 = 320 pixels available for bars.

0 trips → 0 pixels · 200 trips → 160 pixels · 400 trips → 320 pixels

Prepare the data and the drawing area

```
const daily = [
  {date: "Apr 1", rides: 299},
  {date: "Apr 2", rides: 219},
  {date: "Apr 3", rides: 149}
];
```



`width="520" height="240"`

Canvas: 520 × 240 pixels.

Reserve 100 pixels on each side.

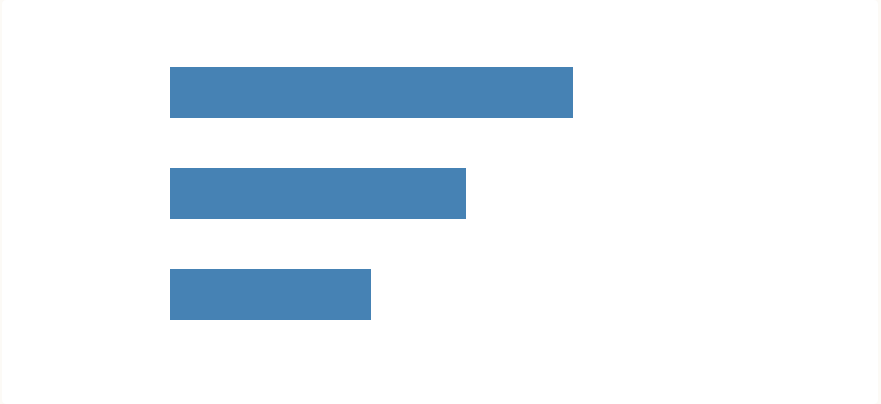
Bar lengths: range([0, 320])

Bars start at `x = 100`; width = `x(day.rides)`, up to 320 pixels.

Row top = `40 + i × 60`; bar height = `30` pixels.

Repeat one rectangle for each record

```
daily.forEach((day, i) => {
  svg.append("rect")
    .attr("x", 100)
    .attr("y", 40 + i * 60)
    .attr("width", x(day.rides))
    .attr("height", 30)
    .attr("fill", "steelblue");
});
```

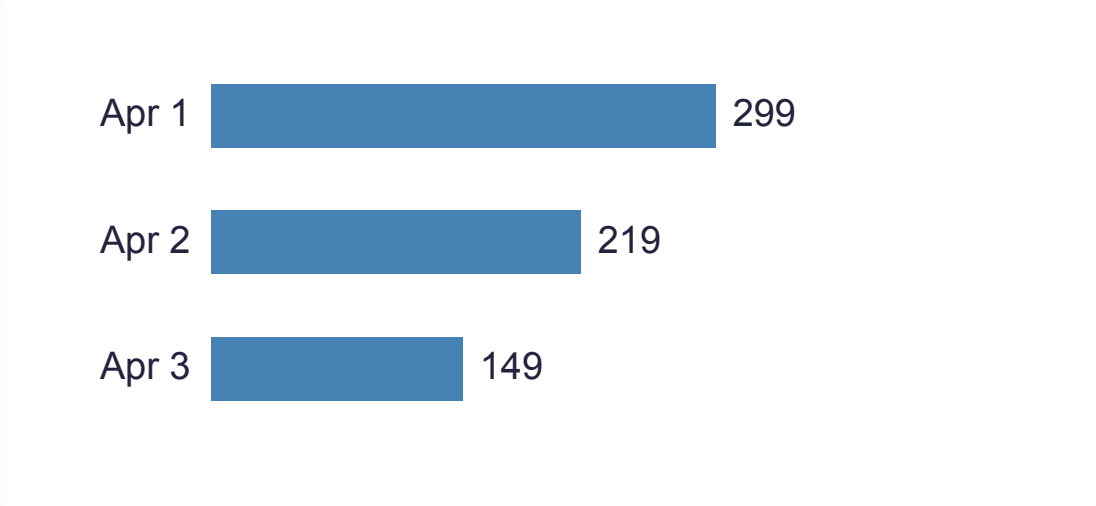


Add the date and count inside the same loop

```
svg.append("text")
  .attr("x", 90).attr("y", 60 + i * 60)
  .attr("text-anchor", "end").text(day.date);
svg.append("text")
  .attr("x", 108 + x(day.rides))
  .attr("y", 60 + i * 60).text(day.rides);
```

Insert these statements inside the `forEach` callback, after the rectangle. The label positions use the same record and index.

The program produces ordinary SVG



Jersey City · April 1–3, 2020 · Citi Bike data

Read the complete project

- `index.html`: the page and SVG.
- `chart.js`: data, scale, and drawing.
- Local D3: reusable library functions.

Open the working chart ↗

Read the complete JavaScript ↗

D3 data binding is another way to build the chart

Besides `forEach`, D3 data binding draws the same bars on a fresh SVG, using the same `daily` array and `x` scale.

Repeat with forEach

```
daily.forEach((d, i) => {
  svg.append("rect")
    .attr("x", 100)
    .attr("y", 40 + i * 60)
    .attr("width", x(d.rides))
    .attr("height", 30)
    .attr("fill", "steelblue");
});
```

Bind with D3

```
svg.selectAll("rect")
  .data(daily)
  .enter().append("rect")
  .attr("x", 100)
  .attr("y", (d, i) => 40 + i * 60)
  .attr("width", d => x(d.rides))
  .attr("height", 30)
  .attr("fill", "steelblue");
```

In D3's attribute callbacks: `d` is the element's bound record; `i` is its index. D3 calls the function for each selected element.

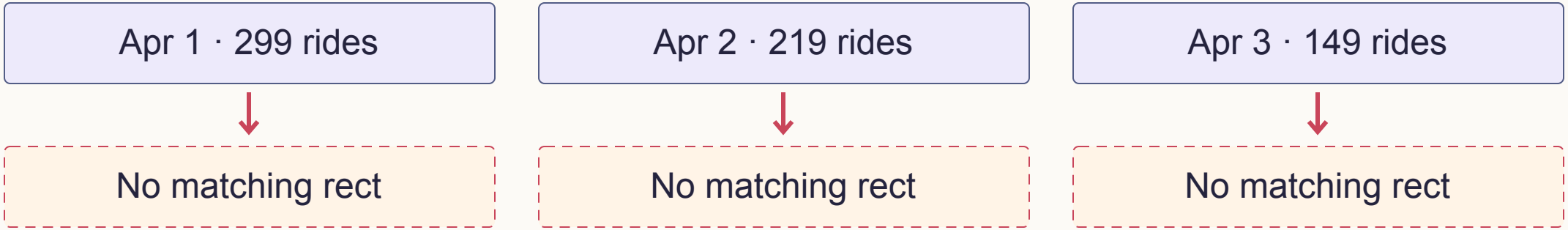
Complete examples: forEach ↗ · D3 data binding ↗ · Download ZIP ↓

data() matches records; it does not create elements

Continue the D3 code from the previous slide, starting with an **empty SVG**. `data(daily)` matches the records against the selected elements.

```
const bars = svg.selectAll("rect").data(daily);
```

`selectAll("rect")` finds **0 rectangles**. All three records have no matching element.



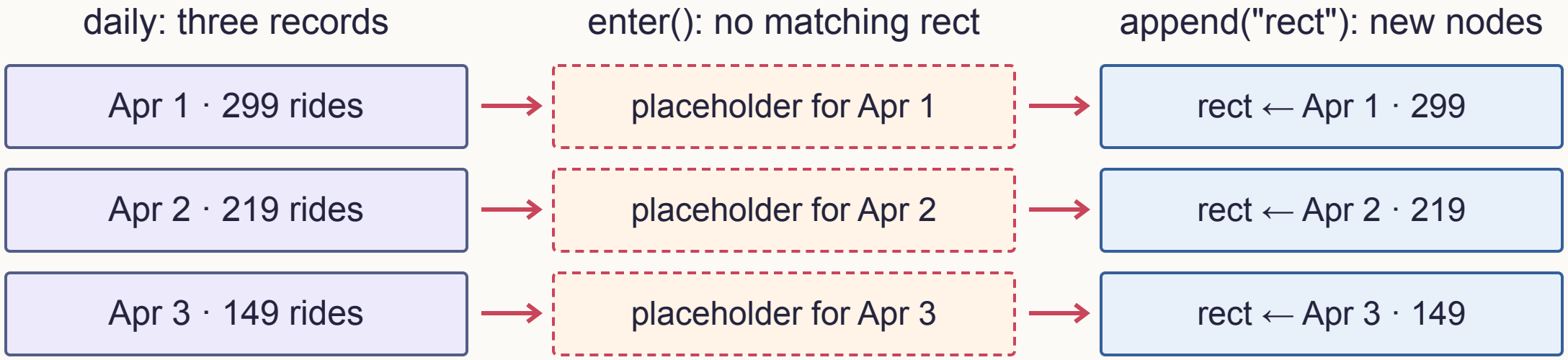
SVG: 0 rectangles before → 0 rectangles after data(daily)

After data(daily): still 0 rect elements. Next, `enter()` gives access to the unmatched records' placeholders; `append("rect")` creates the elements.

enter() identifies records that need elements

Start with an **empty SVG**. `enter()` returns one placeholder for each record without a matching element; `append("rect")` creates one rectangle for each.

```
svg.selectAll("rect").data(daily)
  .enter().append("rect");
```

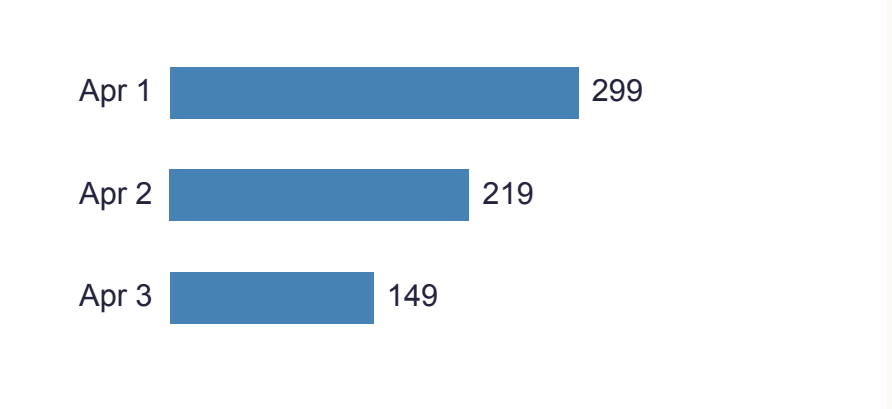


Each new `rect` keeps its record. Set its position, width, and height to make the bar visible.

attr() turns bound data into position and width

The three rectangles now exist and have bound data. Select them and set their SVG attributes:

```
svg.selectAll("rect")
  .attr("x", 100)
  .attr("y", (d, i) => 40 + i * 60)
  .attr("width", d => x(d.rides))
  .attr("height", 30)
  .attr("fill", "steelblue");
```



d: the record bound to this rectangle, such as `{date: "Apr 1", rides: 299}`.

i: its index in this selection: `0`, `1`, or `2`.

Record	i	y	Width
Apr 1 · 299	0	40	239.2
Apr 2 · 219	1	100	175.2
Apr 3 · 149	2	160	119.2

The same linear scale `x` maps **0–400 trips → 0–320 pixels**. D3 calls each function once for each selected rectangle.

Ask AI for one precise, testable change

Task: highlight dates with more than 250 rides.

“In chart.js, use coral for a bar when day.rides > 250; otherwise keep steelblue. Keep the data, labels, positions, and scale unchanged. Explain the lines you changed.”

- Before running: which bar should change?
- After running: check all three colors and count labels.
- Confirm the bar widths and order stay the same.

You can now read the path from data to SVG

- **DOM:** find and change elements in a page.
- **JavaScript:** store records, calculate values, and repeat drawing instructions.
- **D3:** call library functions to create SVG elements and map values to visual properties.

For an AI-generated chart, identify the data, the scale, the loop, and the SVG attributes. Then verify the visible result.